

Relazione tecnica

Esercitazione Python - Calcolo di perimetri e aree di figure geometriche

Corsista: D'Angelo Giovanni

Ambiente di lavoro: Kali Linux

Linguaggio utilizzato: Python

1. Obiettivo dell'esercitazione

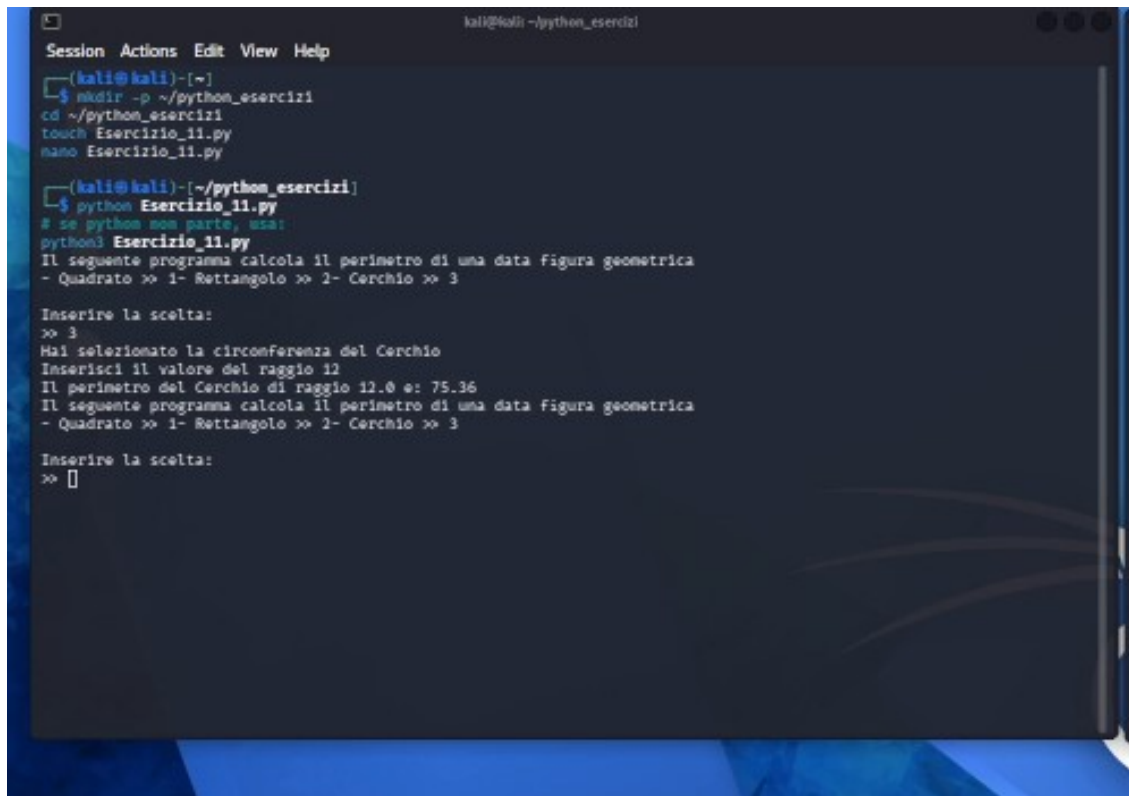
Durante questa esercitazione ho realizzato e testato due programmi Python. Nel primo programma ho implementato una scelta tra diverse figure geometriche per calcolare il perimetro del quadrato, del rettangolo e del cerchio. Nel secondo programma, corrispondente alla parte facoltativa, ho esteso la logica per calcolare sia perimetro sia area, riutilizzando automaticamente il valore dell'area ottenuta come nuovo valore iniziale per la figura successiva.

L'attività è stata svolta interamente in ambiente Kali Linux, utilizzando il terminale per la gestione dei file e l'editor nano per la scrittura del codice Python.

2. Preparazione dell'ambiente di lavoro

Per iniziare ho aperto il terminale e ho predisposto una cartella dedicata agli esercizi Python. All'interno di questa cartella ho creato i due file di lavoro "Esercizio_11.py" e "Esercizio_11_facoltativo.py", così da separare in modo ordinato la versione base da quella facoltativa del programma.

- Creazione della directory di lavoro ~/python_esercizi.
- Creazione del file Esercizio_11.py per il programma base.
- Creazione del file Esercizio_11_facoltativo.py per la versione estesa.
- Apertura dei file con nano per l'inserimento del codice.
- Esecuzione dei programmi dal terminale con il comando python nome_file.py.



```
kali@kali: ~/python_esercizi
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/python_esercizi
$ cd ~/python_esercizi
$ touch Esercizio_11.py
$ nano Esercizio_11.py

(kali@kali)-[~/python_esercizi]
$ python Esercizio_11.py
# se python non parte, usa:
python3 Esercizio_11.py
Il seguente programma calcola il perimetro di una data Figura geometrica
- Quadrato >> 1- Rettangolo >> 2- Cerchio >> 3

Inserire la scelta:
>> 3
Hai selezionato la circonferenza del Cerchio
Inserisci il valore del raggio 12
Il perimetro del Cerchio di raggio 12.0 e: 75.36
Il seguente programma calcola il perimetro di una data Figura geometrica
- Quadrato >> 1- Rettangolo >> 2- Cerchio >> 3

Inserire la scelta:
>> 
```

Figura 1 - Terminale Kali con la cartella di lavoro, l'esecuzione del programma base e il risultato ottenuto per il cerchio.

3. Sviluppo del programma base

Nel file Esercizio_11.py ho definito una funzione principale dedicata al calcolo del perimetro. Il programma presenta inizialmente un messaggio descrittivo e propone all'utente tre opzioni numeriche: 1 per il quadrato, 2 per il rettangolo e 3 per il cerchio. La scelta inserita viene letta tramite `input()` e convertita in intero con `int()`.

Per gestire i diversi casi ho utilizzato una struttura `if-elif-else`. In questo modo il programma esegue il ramo corretto in base alla figura selezionata dall'utente.

- Nel caso del quadrato, il programma richiede il lato e calcola il perimetro con la formula $\text{lato} * 4$.
- Nel caso del rettangolo, il programma richiede base e altezza e calcola il perimetro con la formula $\text{base} * 2 + \text{altezza} * 2$.
- Nel caso del cerchio, il programma richiede il raggio e calcola la circonferenza con la formula $2 * r * 3.14$.
- Nel ramo finale `else`, il programma intercetta un'eventuale scelta non valida.

4. Verifica del programma base

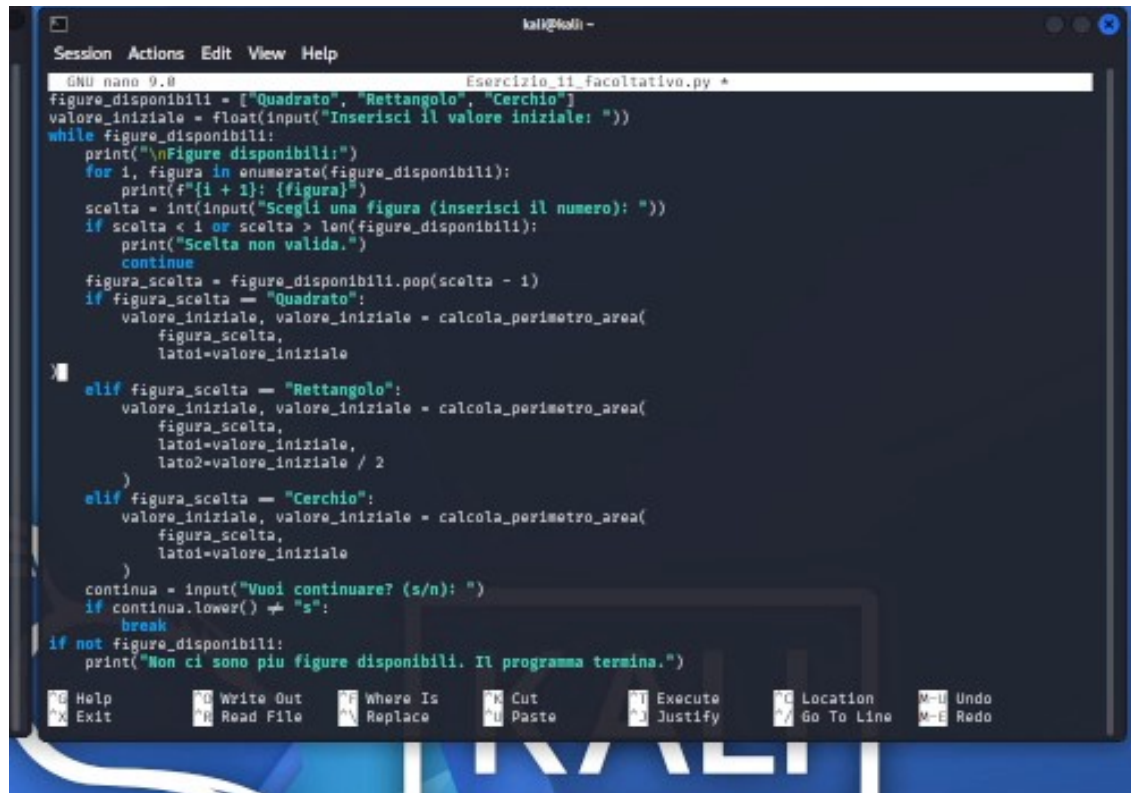
Dopo il salvataggio del file ho eseguito il programma dal terminale con il comando `python Esercizio_11.py`. Durante il test ho selezionato l'opzione 3, corrispondente al cerchio, e ho inserito come raggio il valore 12.

Il programma ha riconosciuto correttamente la selezione, ha stampato il messaggio relativo alla circonferenza del cerchio e ha restituito come risultato il valore 75.36. Questo output è coerente con la formula implementata nel codice, perché $2 * 12 * 3.14$ produce proprio 75.36.

5. Sviluppo del programma facoltativo

Successivamente ho creato il file `Esercizio_11_facoltativo.py` per realizzare la versione avanzata. In questo secondo script ho introdotto una gestione più articolata del problema, includendo sia il calcolo del perimetro sia quello dell'area e permettendo l'esecuzione consecutiva di più scelte geometriche a partire da un solo valore iniziale.

Per organizzare meglio il codice ho utilizzato funzioni separate. Ho importato il modulo `math` e ho definito una funzione specifica per il cerchio, oltre a una funzione generale capace di gestire quadrato, rettangolo e cerchio in base al nome della figura.



```
GNU nano 9.0 Esercizio_11_facoltativo.py *
figure_disponibili = ["Quadrato", "Rettangolo", "Cerchio"]
valore_iniziale = float(input("Inserisci il valore iniziale: "))
while figure_disponibili:
    print("\nFigure disponibili:")
    for i, figura in enumerate(figure_disponibili):
        print(f"{i + 1}: {figura}")
    scelta = int(input("Scegli una figura (inserisci il numero): "))
    if scelta < 1 or scelta > len(figure_disponibili):
        print("Scelta non valida.")
        continue
    figura_scelta = figure_disponibili.pop(scelta - 1)
    if figura_scelta == "Quadrato":
        valore_iniziale, valore_iniziale = calcola_perimetro_area(
            figura_scelta,
            lato1=valore_iniziale
        )
    elif figura_scelta == "Rettangolo":
        valore_iniziale, valore_iniziale = calcola_perimetro_area(
            figura_scelta,
            lato1=valore_iniziale,
            lato2=valore_iniziale / 2
        )
    elif figura_scelta == "Cerchio":
        valore_iniziale, valore_iniziale = calcola_perimetro_area(
            figura_scelta,
            lato1=valore_iniziale
        )
    continua = input("Vuoi continuare? (s/n): ")
    if continua.lower() != "s":
        break
if not figure_disponibili:
    print("Non ci sono più figure disponibili. Il programma termina.")
```

Figura 2 - Porzione del codice della versione facoltativa aperta in nano.

La logica implementata nella versione facoltativa è stata la seguente:

1. Acquisizione di un valore iniziale unico all'avvio del programma.
2. Creazione di una lista chiamata `figure_disponibili` contenente Quadrato, Rettangolo e Cerchio.
3. Visualizzazione dinamica delle figure ancora disponibili mediante un ciclo `while`.
4. Rimozione della figura già scelta con il metodo `pop()`, in modo da non riproporla nei passaggi successivi.
5. Riutilizzo dell'area appena calcolata come nuovo valore iniziale per il calcolo successivo.
6. Interruzione del ciclo quando non restano più figure disponibili oppure quando l'utente decide di non proseguire.

In questa versione ho quindi applicato più costrutti fondamentali del linguaggio Python: funzioni, liste, condizioni `if-elif-else`, ciclo `while`, ciclo `for` per l'elenco numerato delle figure e `input` dell'utente convertiti in `float` o `int` a seconda della necessità.

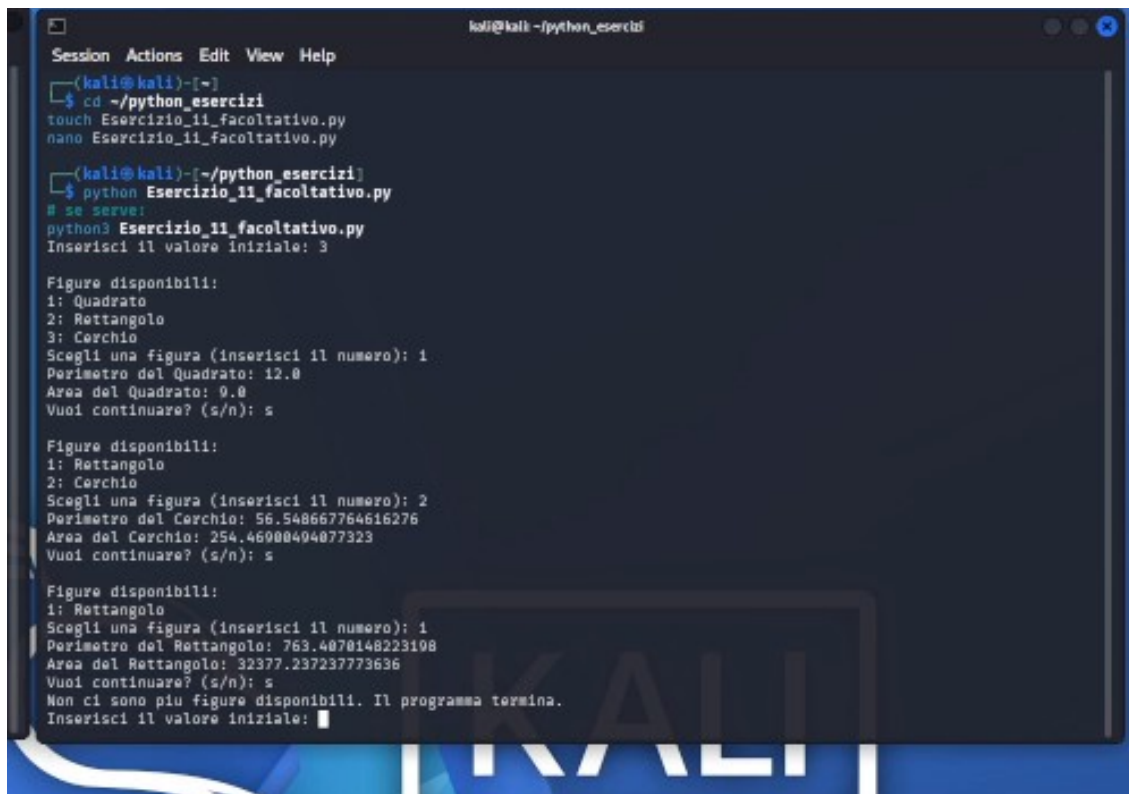
6. Verifica del programma facoltativo

Per il collaudo del secondo script ho eseguito il comando `python Esercizio_11_facoltativo.py`. All'avvio ho inserito come valore iniziale il numero 3.

Nel primo passaggio ho selezionato il quadrato. Il programma ha calcolato correttamente il perimetro del quadrato pari a 12.0 e l'area pari a 9.0, usando come lato il valore iniziale 3.

Nel secondo passaggio ho continuato l'esecuzione e, tra le figure rimaste disponibili, ho selezionato il cerchio. In questo caso il programma ha usato come nuovo valore iniziale il risultato dell'area precedente e ha prodotto un perimetro pari a 56.548667764616276 e un'area pari a 254.46900494077323.

Nel terzo passaggio ho proseguito nuovamente e ho selezionato il rettangolo, unica figura rimasta disponibile. Il programma ha calcolato un perimetro pari a 763.4070148223198 e un'area pari a 32377.237237773636. Al termine dell'ultima elaborazione il software ha comunicato che non erano più presenti figure disponibili e ha concluso l'esecuzione.



```
kali@kali:~/python_esercizi
Session Actions Edit View Help
(kali@kali)~$ cd ~/python_esercizi
touch Esercizio_11_facoltativo.py
nano Esercizio_11_facoltativo.py

(kali@kali)~/python_esercizi$ python Esercizio_11_facoltativo.py
# se serve:
python3 Esercizio_11_facoltativo.py
Inserisci il valore iniziale: 3

Figure disponibili:
1: Quadrato
2: Rettangolo
3: Cerchio
Scegli una figura (inserisci il numero): 1
Perimetro del Quadrato: 12.0
Area del Quadrato: 9.0
Vuoi continuare? (s/n): s

Figure disponibili:
1: Rettangolo
2: Cerchio
Scegli una figura (inserisci il numero): 2
Perimetro del Cerchio: 56.548667764616276
Area del Cerchio: 254.46900494077323
Vuoi continuare? (s/n): s

Figure disponibili:
1: Rettangolo
Scegli una figura (inserisci il numero): 1
Perimetro del Rettangolo: 763.4070148223198
Area del Rettangolo: 32377.237237773636
Vuoi continuare? (s/n): s
Non ci sono piu figure disponibili. Il programma termina.
Inserisci il valore iniziale: 
```

Figura 3 - Output finale della versione facoltativa con i tre calcoli eseguiti in sequenza.

7. Risultati ottenuti

Esecuzione	Valori inseriti	Perimetro	Area / note
Programma base	Scelta 3, raggio 12	75.36	Circonferenza del cerchio
Facoltativo - Quadrato	Valore iniziale 3	12.0	Area 9.0
Facoltativo - Cerchio	Valore iniziale 9.0	56.548667764616276	Area 254.46900494077323
Facoltativo - Rettangolo	Valore iniziale 254.46900494077323	763.4070148223198	Area 32377.237237773636

8. Considerazioni finali

L'esercitazione mi ha permesso di consolidare l'uso di Python nella gestione dell'input utente, delle strutture condizionali e delle funzioni. Nella versione base ho verificato il corretto utilizzo di if-elif-else per distinguere le formule da applicare alle diverse figure geometriche. Nella versione facoltativa ho invece lavorato su una logica più completa, sfruttando liste, cicli e funzioni per concatenare più elaborazioni consecutive in maniera automatica.

Dal punto di vista operativo il lavoro è risultato corretto sia nella fase di scrittura del codice sia nella fase di esecuzione. I test effettuati hanno confermato che i programmi rispondono in modo coerente alla traccia proposta e producono risultati compatibili con le formule geometriche implementate.

D'Angelo Giovanni